

Comparative Analysis of Convolutional Neural Network (CNN) Architectures used in image processing with the Vision Transformer (ViT)

1.Introduction

CNNs have been the backbone of computer vision since the innovation of AlexNet (2012). For image processing, it has become a basic choice. CNNs learn patterns efficiently without needing special parameters due to convolutional filters. This filter slides across the input and shares weights at every spatial location. Since then many architectures were developed to increase CNN's efficiency. Some are VGG, GoogLeNet, ResNet, DenseNet, and EfficientNet. Each tries to improve accuracy while minimizing the cost. Most recently, the Vision Transformer (ViT) has challenged whether convolution is necessary, showing a convolution-free, attention-based model that can go head-to-head with CNNs if it is pre-trained on enough data. This work compares all six architectures in terms of their design, complexity, and performance to determine their sustainability and constraints.

2. Architectural design of the models

2.1 VGG-16 (Simonyan & Zisserman, 2014)

This spatial reduction process belongs to the ConvNet configuration family. VGG-16 uses a 3x3 convolution filter, which is layered repeatedly and separated by 2x2 max-pooling. After each max-pooling, the number of channels doubles. The input image has 64 channels, and the output has 512. Afterwards, it is put through three fully connected layers and a softmax classifier for the final result.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256	conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512	conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512	conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table1: VGG Configuration table showing conv layer stack, Simonyan & Zisserman (2015), p.3

2.2 GoogLeNet (Inception v1) - (Szegedy et al., 2014)

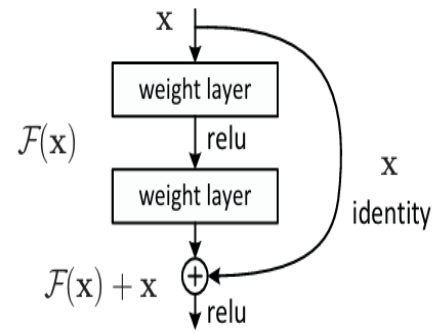
GoogLeNet uses inception module, which is stacked to design a 22 layer network. GoogLeNet takes input image and processes it in parallel by four branches: 1x1 → 1x1 → 3x3 → 5x5 convolution layers. This block then goes through maxpooling (3x3) and another 1x1 convolution (to reduce bottleneck). All outputs are concatenated and passed to the next module.

It is far more parameter efficient than VGG. Only limitation is it has multiple parallel branches and two auxiliary classifiers that is harder to modify than our previous models' uniform design.

2.3 ResNet-50 (He et al., 2015)

A caveat is adding more layers to a network brings more training error. ResNet-50 aims to solve this degradation problem. The best part of it is skip (residual) connection. The work focuses on learning residuals $F(x)$ instead of the entire map. The $F(x)$ is added back to input via identity shortcut without bringing in parameters. ResNet training is

highly reliable (77% with 22.6M params) due to bottleneck use. This is even more accurate and cheaper than VGG. The same idea has been used in networks with 100 to even 1000 layers for training successfully. *Figure1: He et al. (2015), p.2*



2.4 DenseNet - 121 (Huang et al., 2016)

DenseNet-121 doesn't only depend on the previous layer's output; every layer inside the dense block receives feature maps concatenated from the preceding layers. Then it passes its own output forward. This architecture is good for feature reuse and improved gradients because every layer is connected to the loss function and the original input. DenseNet-121 reaches higher competitive accuracy (74.98%) for around 8 million parameters. This is a similar result to ResNet for only a third of its parameter. The only drawback is the cost of higher memory use during training.

2.5 EfficientNet-B0 (Tan & Le, 2019)

This is the baseline network of the EfficientNet family. EfficientNet-B0 is obtained by multi-objective neural architecture search that optimizes accuracy and FLOPs. Its backbone is composed of MBConv blocks, which are mobile inverted bottleneck convolutions. Its main contribution is not the block itself but the compound scaling. It scales depth, width, and resolution combined using a single coefficient instead of doing all three separately. This gives accuracy of around 77% with only 5.3 million parameters. This is so far the best computation-to-accuracy ratio of the architectures mentioned here. The same scaling rule is used for higher-up variants, proving even better accuracy results.

2.6 Vision Transformer (ViT-B/16) - (Dosovitskiy et al., 2020)

Vision Transformer completely abandons the convolution process. Here, the image is split into 16×16 patches with linear embedding. This is processed by a standard transformer encoder with

the help of a global transformer process. The process is done across all patches. ViT learns spatial relationships from data directly. This is unlike CNNs, which have image-specific biases like locality. This architecture has around 86 million parameters and needs large amounts of pre-training data to perform well. Without that training, it reaches around CNN accuracy (~76%), but with training it goes up to 81% accuracy.

3. Key innovations

Architecture	Key Innovation	Main Benefit
VGG-16	Depth via uniform, stacked 3x3 convolutions	Simplicity, strong transfer features
GoogLeNet	Inception module: Parallel multi-scale convolutions + 1x1 bottlenecks	Multi-scale features, parameter efficiency
ResNet-50	Residual (identity shortcut) connection	Enables very deep, stable training
DenseNet-121	Dense connectivity via feature concatenation	Maximum feature reuse, fewer parameters
EfficientNet-B0	Compound scaling of depth/width/resolution	Best accuracy-per-FLOP
ViT-b/16	Global self-attention over image patches, no convolution	Scales extremely well with more data, especially if pre-trained

4. Computational complexity and Performance

The general performance trend of these models is a steady increase in efficiency over time. We will see in the table below that EfficientNet-B0 gives equal or better results than ResNet using only a fifth of its parameters and a tenth of the FLOPs (computational cost). Again GoogLeNet and DenseNet-121 sit below ResNet in performance trends. ViT-B/16 doesn't follow the trends of parameter/FLOPs alone. Its benefits can be only seen when using large datasets.

Architecture	Params(M)	FLOPs(G)	Top-1 Acc(%)
VGG-16	138	15.3	74.4
GoogLeNet	6.8	1.5	~71.5
ResNet-50	25.6	3.8	77.15
DenseNet-121	8	2.9	74.98
EfficientNet-B0	5.3	0.39	77.1
ViT-B/16	86	17.6	77.91

			84.15 if pre-trained
--	--	--	----------------------

5. Suitability in Image tasks and limitations

Architecture	Best Suited For	Main Benefit
VGG-16	Transfer learning, feature extraction, and baseline use.	Simple structure but heavy parameter use, making it slow
GoogLeNet	Multi-scale recognition on a moderate compute budget	Good ratio of accuracy to parameters, but complicated design. Makes it harder to trace back
ResNet-50	In general classification, framework of detection/segmentation	Strong, stable accuracy. But still not much improvement in parameter count
DenseNet-121	Smaller dataset where overfitting can be a risk	This one is efficient in parameters, but requires more training memory. Makes cost higher
EfficientNet-B0	Mobile/embedded deployment	Best accuracy-per-FLOP, scalable via compound coefficient
ViT-b/16	Large-scale pre-training pipelines	Best performance if pre-trained, poor performance if small dataset

Conclusion:

Looking at all the architectures, it is clear that every next innovation is a solution to the limitation of previous work. It is not an entirely separate concept. Earlier models like VGG-16 relied on raw depth to improve accuracy but with high cost. GoogLeNet and ResNet-50 both improves efficiency and accuracy. ResNet becomes a general framework for many other works. DenseNet-121 pushed connectivity even further due to higher parameter efficient CNN. This is useful for smaller datasets. EfficientNet-B0 gives the best accuracy rate for a reasonable computational cost, and becomes well suited for embedded use.

ViT-B/16 breaks this pattern by abandoning the convolution entirely. It can outperform all these models if pre-trained on large scale data. Otherwise it falls in a clear disadvantage. Overall the understanding is that there is no clear best work, use of these models depend on available resources. Depending on dataset size, computation cost, output needs - the preference of models will keep changing.

References

- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv:2010.11929 [Cs]*. <https://arxiv.org/abs/2010.11929v2>
- Huang, G., Liu, Z., & Weinberger, Kilian Q. (2016). *Densely connected convolutional networks*. arXiv.Org. <https://arxiv.org/abs/1608.06993>
- Kaiming, H., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition*. <https://doi.org/10.48550/arXiv.1512.03385>
- Simonyan, K., & Zisserman, A. (2015, April 10). *Very deep convolutional networks for large-scale image recognition*. arXiv.Org. <https://arxiv.org/abs/1409.1556>
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2014). *Going deeper with convolutions*. arXiv.Org. <https://doi.org/10.48550/arXiv.1409.4842>
- Tan, M., & Le, Q. V. (2019, May 28). *EfficientNet: Rethinking model scaling for convolutional neural networks*. arXiv.Org. <https://arxiv.org/abs/1905.11946>